Duplicate Record Detection Using Attribute Normalisation, Blocking and Similarity Fusion

J. Kumudavalli, R. Varshitha, P. Abhinaya

Dept. of Computer Science and Engineering, KL University, KLH Bowrampet Campus, Hyderabad, India
2420030592 · 2420030536 · 2420030309

Abstract—Duplicate and near-duplicate records reduce data quality in almost every real dataset, arising from repeated entry, inconsistent formatting, spelling variation and missing values. Exact matching fails on these records because the values are equivalent in meaning but not in representation. This paper presents a four-stage duplicate detection system combining attribute-aware normalisation, exact matching on a normalised key, redundant multi-key blocking, and a weighted fusion of three string similarity measures computed only over attributes present in both records. The system uses Python, Pandas and NumPy, with all string metrics implemented directly. On a synthetic benchmark of 2,063 records over 1,400 entities containing eleven labelled corruption classes, with the decision threshold selected on a disjoint development split, the system achieves precision 0.9921, recall 0.9783 and F1 0.9852 on held-out data. Blocking removes 92.5% of candidate pairs while retaining 100% of true duplicates, giving a 25.2× speed-up at no cost in F1. Ablation shows normalisation, not the similarity metric, is the dominant contributor.

Keywords—duplicate record detection, entity resolution, data cleaning, blocking, string similarity, data quality

I. Introduction

Datasets are a core organisational resource, but their value is limited by their quality. A common and costly defect is the duplicate record: two or more rows referring to the same real-world entity while differing in stored form. These accumulate through ordinary processes — a customer registering twice with a different spelling, a form submitted with different capitalisation, or a phone number stored once with a country code and once without.

The effects are not cosmetic. Duplicates bias every aggregate computed over the table, degrade machine learning models by placing near-identical rows on both sides of a train–test split, fragment an entity's history across several identifiers, and waste storage. They then propagate into reports used for decisions.

Exact matching fails precisely where the problem is hardest: it treats “Ravi Kumar” and “RAVI KUMAR” as different, and “+91-9876543210” and “9876543210” as different. What is needed is a system that first removes representational variation and then measures residual similarity.

Two difficulties complicate this. Comparing all pairs of n records needs $n(n-1)/2$ comparisons — over two million for only a few thousand rows. Duplicates are also rare among all pairs: in our benchmark they are 0.047% of pairs, so precision is far more fragile than recall.

This paper presents a four-stage pipeline addressing both, and evaluates it with the threshold selected on a held-out split, with an ablation study, four baselines, and a per-corruption breakdown.

II. Related Work

The field has been studied for decades as record linkage, entity resolution and deduplication [1], [3]. Fellegi and Sunter [2] formalised linkage as a probabilistic decision over a comparison vector, which remains the reference model but requires estimating agreement probabilities from labelled data. We use a deterministic weighted rule instead, trading statistical optimality for transparency and no training requirement.

String similarity underpins practical systems. Levenshtein distance [4] models typographical error; the Jaro measure [5] and Winkler's prefix-weighted extension [6] were designed for personal names. Because exhaustive comparison is quadratic, scalable systems use blocking: Hernández and Stolfo [7] introduced sorted-neighbourhood, and Christen [8] surveys indexing families and the tension between reduction ratio and pair completeness.

Recent work applies deep learning to matching [9], but reports the largest gains on dirty textual attributes, while clean structured attributes are already well served by classical methods. This motivates our design: for data whose corruption is mainly formatting and typographical, a well-normalised deterministic pipeline should be strong and remains auditable field by field.

Two gaps motivate this work. Evaluations often report a single F1 at a threshold tuned on the same data, conflating method quality with tuning quality; and aggregate scores hide which error classes a system actually solves. We address both.

III. Proposed Methodology

Let $D = \{r_1,\ldots,r_n\}$ be a dataset where each record is generated by an entity $e(r)$. The ground truth is $M = \{(r_i,r_j) : i<j,\ e(r_i)=e(r_j)\}$, and the task is to recover M from attribute values alone, evaluated by precision, recall and F1. The system is the four-stage pipeline of Fig. 1.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

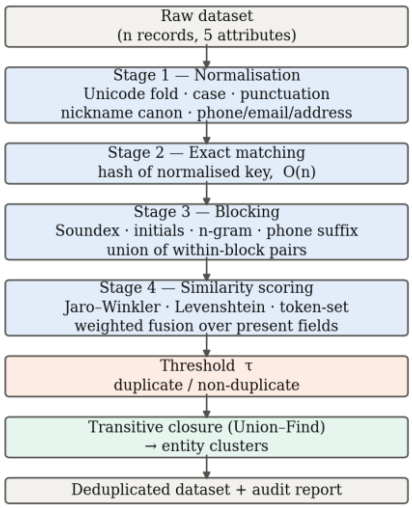


Fig. 1. The four-stage detection pipeline.

A. *Stage 1 — Attribute-Aware Normalisation*

Each attribute is canonicalised by its own failure modes rather than one generic rule. **Names** are Unicode-folded (NFKD, combining marks removed), lowercased, stripped of punctuation, expanded through a nickname lookup, and token-sorted so the form is invariant to surname-first ordering. **Emails** are lowercased, with "+tag" suffixes, dots and trailing digits removed from the local part, keeping the domain separate. **Phone numbers** keep digits only, dropping an international prefix or trunk zero and retaining the last ten digits. **Addresses** are expanded through an abbreviation table.

B. *Stage 2 — Exact Matching*

The concatenated normalised attributes form a key; records sharing it are duplicates by construction. This is one hash pass, O(n). It recovers 0.5716 recall at perfect precision, against 0.0927 for the same operation on raw values — which quantifies Stage 1 on its own.

C. *Stage 3 — Redundant Blocking*

Candidates are restricted to pairs sharing at least one blocking key. Each record gets several keys of deliberately different kinds: the **Soundex** code of each name token (robust to spelling variants), the **sorted initials** (robust to token reordering), **prefixes** of the name and email local part, and the **last seven digits** of the phone number. The candidate set is the union of within-block pairs. Redundancy is the key property: a corruption destroying one key rarely destroys all. Blocks larger than 120 records are discarded as degenerate; only 3 of 1,979 blocks exceeded this.

D. *Stage 4 — Similarity Fusion*

For names, three measures are computed and the maximum taken:

$$\mathrm{sim}(a,b) = \max\left(\mathrm{JW}(a,b),\ \mathrm{TS}(a,b),\ 1 - \mathrm{lev}(a,b)/\max(|a|,|b|) \right) \quad (1)$$

where JW is Jaro–Winkler, TS an order-independent token-set overlap, and lev the Levenshtein distance via a two-row dynamic programme. The maximum acts as a disjunction of evidence, since the three measures fail on disjoint error classes. Phones are compared for exact equality, then digit-suffix agreement, then edit ratio at a discount; emails by Jaro–Winkler on the local part, discounted when domains differ.

Scores are fused over only those attributes C present in *both* records, renormalised by the observed weight mass:

$$S(r_i,r_j) = \Sigma_{a\in C}\ w_a \cdot \mathrm{sim}_a\ /\ \Sigma_{a\in C}\ w_a \quad (2)$$

Scoring a missing value as zero would penalise a record for incompleteness rather than disagreement; (2) judges it only on evidence it carries. A pair is a duplicate when $S \geq \tau$ and at least two attributes were compared. Weights are name 0.34, email 0.26, phone 0.24, address 0.13, city 0.03, reflecting discriminative power. Pairwise decisions are finally merged into entity clusters by Union–Find [10].

IV. Experimental Setup

Public benchmarks rarely publish the error class of each duplicate, preventing per-class analysis. We therefore generate a benchmark where every duplicate is produced by exactly one named corruption, giving ground truth over both pairs and error classes.

The generator produces 1,400 entities over five attributes — name, email, phone, address, city — from vocabularies representative of the Indian context. 30% of entities receive one to three duplicates, giving 2,063 records and 992 true pairs among 2,126,953. Generation is seeded and reproducible. Table I lists the corruption classes.

Class	Pairs	Description
exact	62	identical re-entry
case_whitespace	107	capitalisation, spacing
spelling_typo	82	adjacent-key substitutions
phone_format	70	+91, zero, dashes
missing_fields	72	attributes absent
email_variant	64	dots, tags, provider
nickname	57	short/long name forms
name_order	50	surname first, initials
address_abbrev	50	abbreviated address
combined	30	two classes stacked
unicode	19	diacritics, odd spaces

Ground truth is the full within-entity pair set, so the system is credited for pairs recovered by closure and charged for over-merges. To avoid reporting a threshold tuned on the evaluation data, *entities* are split into a development set (560 entities, 801 records) and a held-out test set (840 entities, 1,262 records), so no duplicate pair straddles the split. τ is chosen to maximise development F1 and applied unchanged to the test set. Implementation is Python 3.11 with Pandas [11] and NumPy [12], single-threaded.

V. Results and Discussion

Fig. 2 sweeps the decision threshold. The curves show the shape typical of a severely imbalanced pair space: recall is saturated over a wide range while precision rises steeply, so the operating point is governed almost entirely by precision.

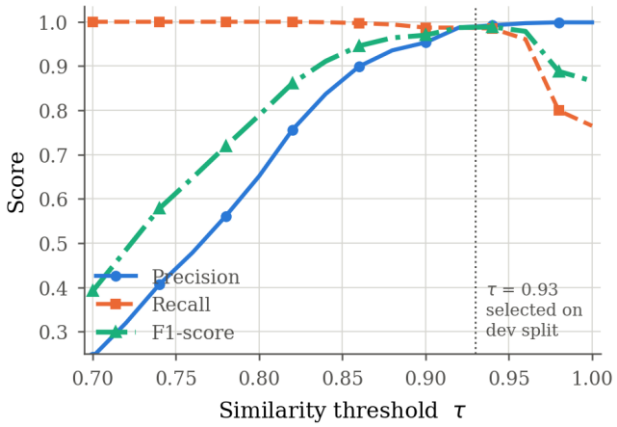


Fig. 2. Precision, recall and F1 against threshold τ.

Maximising development F1 selects τ = 0.93. Table II reports performance at that threshold.

Split	τ	P	R	F1
Development (tuned)	0.93	1.0000	0.9971	0.9986
Held-out test	0.93	0.9921	0.9783	0.9852
Test, oracle τ	0.94	0.9953	0.9783	0.9867
Full dataset	0.93	0.9879	0.9849	0.9864

The gap between the transferred and oracle thresholds on test data is only 0.0015 F1, so the threshold is not sharply peaked and transfers across disjoint entity populations — it can be tuned once and reused rather than re-tuned per dataset.

A. *Blocking Efficiency*

Quantity	Value
All record pairs	2,126,953
Candidate pairs retained	159,913
Reduction ratio	92.48%
Pair completeness	100.00%
Runtime, blocked	15.1 s
Runtime, exhaustive	380.8 s
Speed-up	25.2×
F1, blocked / exhaustive	0.9864 / 0.9864

Redundant keys achieve 100% pair completeness — no true pair is lost — while discarding 92.48% of comparisons. Because completeness is total, blocked and exhaustive runs give identical F1, so the 25.2× speed-up costs no accuracy at all. This follows from key redundancy rather than from any single key being well chosen.

B. *Recall by Corruption Class*

Fig. 3 decomposes recall by the corruption class injected into each duplicate pair.

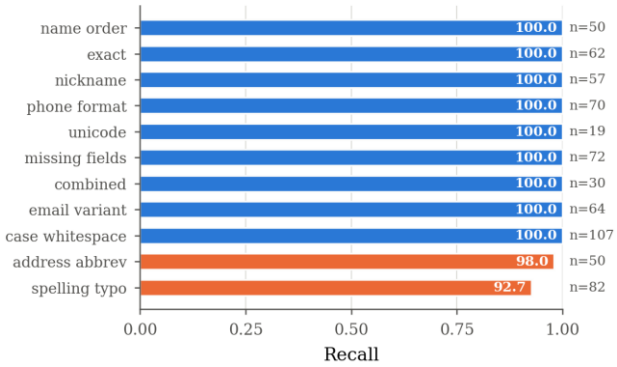


Fig. 3. Recall by corruption class at τ = 0.93.

The system is perfect on nine of eleven classes, including the combined class. Spelling typos reach 0.9268 and address abbreviation 0.9800. The typo case is informative: when two adjacent-key substitutions land in a short given name, the corrupted token can be closer to an unrelated name than to its source. That is a real limit of character-level matching, not a tuning artefact.

C. *Baselines and Ablation*

Method	P	R	F1
Exact match on raw attributes	1.0000	0.0927	0.1697
Exact match after normalisation	1.0000	0.5716	0.7274
Levenshtein on concatenated record	1.0000	0.8810	0.9368
Jaro-Winkler on name only	0.1085	0.8276	0.1918
Proposed system	0.9879	0.9849	0.9864

Exact matching on raw attributes recovers only 0.0927 recall. Normalising first raises it to 0.5716 at no precision cost, so normalisation alone resolves most duplicates here. Edit distance over the concatenated record is a strong baseline at F1 0.9368, but loses 10.4 recall points because one long string dilutes a strong disagreement in one field. Name-only matching collapses to F1 0.1918 — recall stays high but precision falls to 0.1085, because distinct people share names. That is the clearest evidence that multi-attribute evidence, not better name matching, makes the problem tractable.

Configuration	P	R	F1
Full system	0.9879	0.9849	0.9864
No normalisation (all fields)	0.9912	0.6774	0.8048
No name normalisation	0.9871	0.7742	0.8678
No email normalisation	0.9923	0.7802	0.8736
No phone normalisation	0.9866	0.8921	0.9370
Uniform field weights	0.9899	0.9929	0.9914
Name attribute only	0.1049	0.8669	0.1871

Normalisation dominates: removing it costs 30.7 recall points, more than any other component. Removing it per attribute is proportionate — names cost 21.1 points and email 20.5, while phone costs only 9.3, because the numeric blocking key and digit-suffix comparison give redundant paths to the same decision.

One result contradicts our design. Uniform weights *improve* F1 from 0.9864 to 0.9914. Because (2) renormalises over present attributes, and blocking has already removed the pairs where weighting would matter, the weights have little left to do. We report this rather than silently adopting uniform weights: on this benchmark the weighting is not doing the work we attributed to it.

D. *Entity-Level Results and Errors*

After closure the system reduces 2,063 records to 1,396 entities against a true 1,400, recovering 403 of 420 multi-record clusters exactly (96.0%). The slight under-count reflects over-merges, which closure amplifies: one false-positive pair can fuse two correct clusters. This argues for a higher threshold whenever a wrong merge is costlier than a missed one.

Every high-confidence false positive shares one structure: two distinct entities with identical normalised names where the discriminating attribute is missing in one record. Since (2) renormalises over present attributes, such a pair is judged on name and city alone. This is an irreducible limit rather than a scoring defect — the records do not contain the evidence

needed to separate them, and a human reviewer given the same rows could not separate them either.

VI. Conclusion and Future Work

We presented a four-stage duplicate detection pipeline combining attribute-aware normalisation, exact matching, redundant blocking and missing-value-aware similarity fusion. On a benchmark of 2,063 records with eleven labelled corruption classes, with τ selected on an entity-disjoint development split, it achieved precision 0.9921, recall 0.9783 and F1 0.9852 on held-out data, recovering 96.0% of clusters exactly.

Three findings stand out. Normalisation, not the similarity metric, is dominant, worth 30.7 recall points. Redundant blocking gives a 25.2× speed-up at zero accuracy cost. And the weighting scheme did not improve on uniform weights — a negative result that bears on where effort should be spent.

The main limitation is that the benchmark is synthetic: its corruptions are rule-generated, and real data contains error modes not modelled here, so these scores should be read as an upper bound. Future work should validate on a public real-world benchmark, learn the threshold and weights from labelled pairs rather than setting them by hand, and apply a learned or transformer-based matcher selectively to the narrow band of uncertain pairs.

References

[1] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate record detection: A survey," IEEE Trans. Knowl. Data Eng., vol. 19, no. 1, pp. 1–16, Jan. 2007.

[2] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," J. Amer. Statist. Assoc., vol. 64, no. 328, pp. 1183–1210, Dec. 1969.

[3] P. Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Berlin, Germany: Springer, 2012.

[4] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[5] M. A. Jaro, "Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida," J. Amer. Statist. Assoc., vol. 84, no. 406, pp. 414–420, Jun. 1989.

[6] W. E. Winkler, "String comparator metrics and enhanced decision rules in the Fellegi–Sunter model of record linkage," in Proc. Section on Survey Research Methods, Amer. Statist. Assoc., 1990, pp. 354–359.

[7] M. A. Hernández and S. J. Stolfo, "The merge/purge problem for large databases," in Proc. ACM SIGMOD Int. Conf. Management of Data, 1995, pp. 127–138.

[8] P. Christen, "A survey of indexing techniques for scalable record linkage and deduplication," IEEE Trans. Knowl. Data Eng., vol. 24, no. 9, pp. 1537–1555, Sep. 2012.

[9] S. Mudgal et al., "Deep learning for entity matching: A design space exploration," in Proc. ACM SIGMOD Int. Conf. Management of Data, 2018, pp. 19–34.

[10] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," J. ACM, vol. 22, no. 2, pp. 215–225, Apr. 1975.

[11] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 56–61.

[12] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, Sep. 2020.